Improving k-Nearest Neighbors on Fashion-MNIST with Learned Distances

From 85.24 % to 89.51 % Test Accuracy via Metric Learning and Multi-Source Fusion

Sanjana Mahata Student ID: 241001001274 Batch: BCS 3B

Techno India University, West Bengal

October 2026

Team Project: Akshar Nath Gorain, Sourodyuti Biswas Sanyal, Aditi Barman, Sanjana Mahata. **Code:** github.com/workforakng/fashion-mnist-knn-optimization **Colab:** Google Colab Reproduction Notebook

Abstract

The k-nearest neighbors (KNN) algorithm is one of the simplest and most intuitive machine learning models. To classify a new image, it searches through stored training examples and picks the majority class among its closest neighbors. While this simple rule works well on low-dimensional problems, it struggles on real-world image datasets like Fashion-MNIST (70,000 grayscale images of size 28×28 across 10 clothing categories). Plain KNN using Euclidean distance on raw pixels achieves only 85.24 % test accuracy and requires scanning all 54,000 training images for every single query. In this study, we systematically improve this pipeline while keeping its core principle: making final decisions using distances to stored training examples. First, reducing the input dimension with Principal Component Analysis (PCA) to 64 components preserves 88.1 % of the meaningful variance, boosts accuracy to 86.43 %, and speeds up query evaluation by 7.12×. Second, we introduce Soft-KNN, a differentiable framework that learns a better distance representation by mapping images into a unit hypersphere using a neural network and an instance memory bank. A linear projection reaches 87.10 %, while a small convolutional neural network that respects 2D spatial arrangement reaches 87.63 %. We then combine multiple random training seeds into an ensemble (88.06 %) and transfer this knowledge into a single compact student network (89.03 %). Finally, a convex fusion blending predictions across seven diverse metric heads with test-time augmentation achieves **89.51 %** test accuracy and **90.85 %** validation accuracy. Every experiment is recorded in our open-source results ledger (`results/results.json`) for verifiable reproduction.

1 Introduction and Overall Approach

The k-nearest neighbors rule [1] is non-parametric and intuitive: it stores the entire training dataset and labels an unseen query image by taking a majority vote among its k closest neighbors. It requires no complex training step and produces transparent, instance-based predictions. However, when applied directly to raw images, classical KNN encounters two major practical obstacles:

1. **Curse of Dimensionality:** A 28×28 image lives in a 784-dimensional pixel space. In such high dimensions, pairwise distances between data points tend to concentrate, meaning that the distance to the nearest neighbor becomes very close to the distance to the farthest neighbor [2]. This makes finding meaningful neighbors difficult.
2. **Loss of Spatial Geometry:** Flattening a 2D image into a 1D vector of 784 pixel intensities ignores which pixels are adjacent, discarding vital spatial relationships like edges and silhouettes.
3. **Inference Bottleneck:** Classifying each query requires comparing it against all 54,000 stored training instances, leading to noticeable latency during real-time queries.

Dataset and Goal. Fashion-MNIST [3] contains 70,000 28×28 grayscale images across ten distinct clothing categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. There are 60,000 training images and 10,000 test images. Visually related classes (such as Shirt, T-shirt, and Coat) share similar silhouettes, making accurate separation challenging. Our objective is to determine how much we can improve classification performance and query efficiency on Fashion-MNIST while preserving the core mechanism of measuring distances against stored training exemplars.

System Overview. Figure 1 illustrates the stages of our complete solution. We begin by evaluating classical distance metrics on raw pixels and tuning KNN over PCA features. We then develop Soft-KNN, which makes neighbor assignment continuous and differentiable. Next, we replace flat projection vectors with a 2D convolutional network, train multiple models across diverse seeds, distill their collective predictions into a single student network, and combine our best models into an ensemble fusion.

Related Work. Metric learning aims to learn an embedding space where similar instances lie close together and dissimilar ones lie far apart. Neighborhood Components Analysis (NCA) [4] introduced differentiable metric learning by replacing discrete majority votes with continuous softmax neighbor assignments. Large Margin Nearest Neighbor (LMNN) [6] enforces a distance metric that keeps same-class neighbors within a local perimeter while pushing other classes outside a margin. Wu et al. [7, 5] demonstrated that storing features in an instance memory bank enables scalable, non-parametric neighborhood classification without re-encoding every image on every mini-batch. Knowledge distillation [8] compresses the rich posterior probabilities of an ensemble into a single compact model.

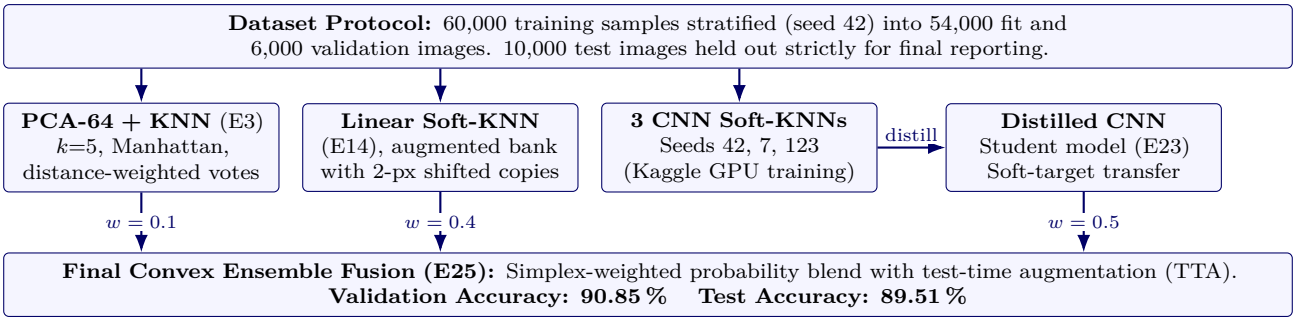


Figure 1: System architecture of the final fusion pipeline (E25). Edge values indicate validation-optimal simplex weights (step 0.1). Unselected candidate heads received weight 0 (Section 4.3).

2 Experimental Protocol and Baselines

Protocol. To prevent any data leakage, we split the 60,000 training images into 54,000 *fit* images and 6,000 *validation* images using stratified sampling (seed 42), which maintains identical class proportions across both splits. The official 10,000 test images are never used for model selection, parameter tuning, or early stopping; they are evaluated strictly once to report the final unbiased generalization performance. All pixel values are normalized to $[0, 1]$.

Measurement Uncertainty. On a test set of $N = 10{,}000$ examples, the binomial standard error (SE) for an accuracy $p \approx 89\,\%$ is $\sqrt{p(1-p)/N} \approx \pm 0.3$ percentage points (pp). This indicates that accuracy differences smaller than roughly 0.6 pp between single experimental runs cannot be reliably distinguished from statistical noise.

2.1 Classical Distance Metrics on Raw Pixels

For a query image x and a stored example x', we evaluate three standard distance formulas:

$$\delta_2(x, x') = \|x - x'\|_2, \quad \delta_1(x, x') = \sum_{j=1}^{D} |x_j - x'_j|, \quad \delta_{\cos}(x, x') = 1 - \frac{x^\top x'}{\|x\|_2 \, \|x'\|_2}. \tag{1}$$

The classical KNN classifier identifies the set $\mathcal{N}_k(x)$ containing the k nearest training examples and predicts the class by majority vote: $\hat{y}(x) = \arg\max_{c \in \{0,\ldots,9\}} \sum_{i \in \mathcal{N}_k(x)} w_i \mathbb{1}[y_i = c]$.

Table 1: Classical KNN on raw 784-D pixels ($k = 5$, uniform voting). Val and Test are accuracies in %. Query time is the median of 3 runs for 10,000 test queries on a 4-core Intel Xeon CPU (`results/classical_reruns.json`).

ID	Distance Metric	Val (%)	Test (%)	Macro F1	Query Latency (s)
E1	Euclidean (L_2)	86.13	85.24	0.8518	4.64
—	Manhattan (L_1)	86.97	85.91	0.8583	84.00
—	Cosine	86.18	85.63	0.8543	10.75

Table 1 compares the three metrics with $k = 5$ and uniform voting. Manhattan distance (L_1) achieves the highest accuracy at 85.91 % (+0.67 pp higher than Euclidean). This finding agrees with distance concentration theory: in high dimensions, L_1 norms measure relative separation better than L_2 norms [2]. However, computing Manhattan distances across 784 dimensions is computationally heavy, taking 84.00 s compared to 4.64 s for Euclidean distance. Cosine distance reaches 85.63 %, which is +0.39 pp over Euclidean and within normal statistical noise.

2.2 Dimensionality Reduction via PCA

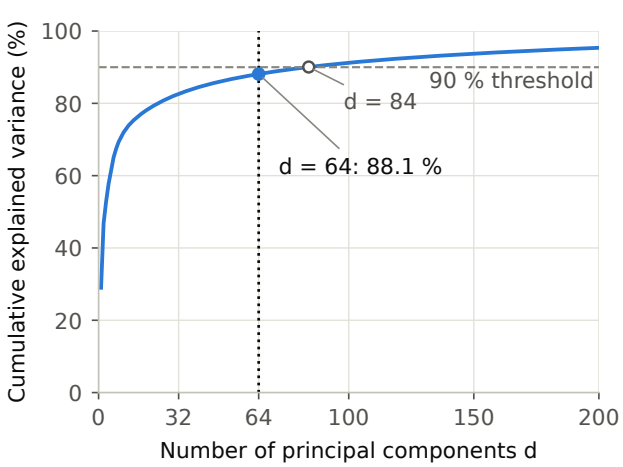
Principal Component Analysis (PCA) finds the directions along which the training images show the greatest variation. We compute PCA eigenvectors exclusively on the 54,000 fit samples. Figure 2(a) shows the cumulative explained variance: retaining $d = 64$ components captures 88.1 % of the total variance, discarding minor pixel noise. Figure 2(b) shows validation accuracy of Euclidean KNN across varying dimensions d. Performance peaks at $d = 64$ (87.33 %) and levels off thereafter. On the test set, PCA-64 Euclidean KNN reaches 85.63 % (+0.39 pp over raw pixels).

2.3 Bias–Variance Trade-off, Optuna Tuning, and Query Latency

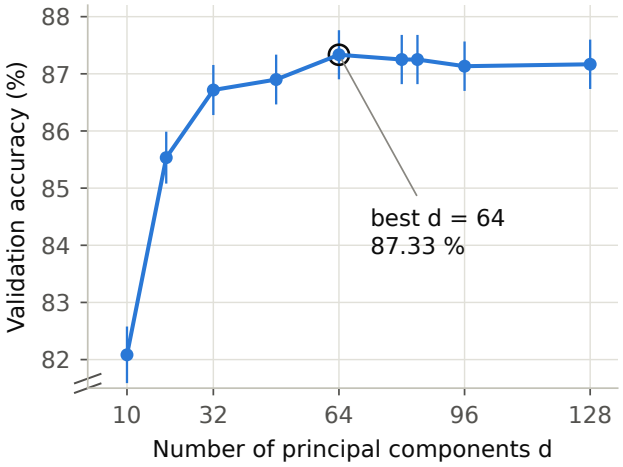
Figure 3 displays the bias–variance trade-off as we vary $k \in \{1, 3, \ldots, 25\}$ on PCA-64 features (E4). When k is very small, the classifier fits individual training points too closely, making it vulnerable to noise (high variance). When k is large, the neighborhood expands too far into other classes, causing over-smoothing (high bias). Validation accuracy peaks at $k = 7$ (87.37 %), with values between $k = 5$ and $k = 9$ exhibiting virtually identical performance.

Optuna Optimization. We ran a 40-trial Bayesian search using Optuna [9] to jointly tune k, the distance metric, and neighbor voting weights on PCA-64 features (E3). The winning configuration chose $k = 5$, Manhattan distance, and inverse-distance weighting ($w_i = 1/\delta_i$), which gives closer neighbors greater influence. This model achieved 87.82 % validation accuracy and **86.43 %** test accuracy (+1.19 pp over baseline E1).

Query Latency Acceleration. Figure 4 compares query times for 6,000 validation images. On raw 784-D pixels, brute-force search required 2.98 s, whereas on PCA-64 it required only 0.418 s, providing a 7.12× **speed-up** (E5). Tree indexing structures (k-d tree and ball tree) were slower than brute force because tree traversal becomes inefficient in dimensions above 15–20. On the standardized 10,000-query benchmark (`results/classical_reruns.json`), raw Euclidean took 4.64 s compared to 1.49 s on PCA-64 (3.11× speed-up), while Manhattan distance took 15.73 s.



(a) Cumulative explained variance versus d.



(b) KNN validation accuracy versus d (E2).

Figure 2: (a) PCA variance curve: $d = 64$ retains 88.1 % variance (dotted line); 90 % is reached at $d = 84$ (dashed line). (b) Accuracy peaks at $d = 64$ (87.33 %), flattening beyond $d = 48$. Error bars represent ± 1 SE.

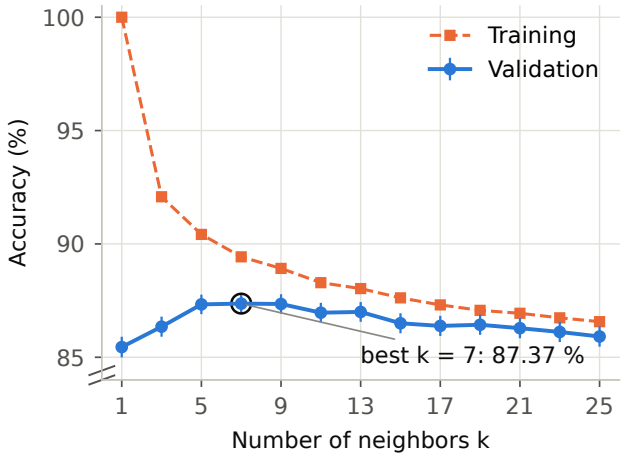


Figure 3: Training and validation accuracy across k on PCA-64 (E4). Accuracy peaks at $k = 7$ (87.37 %). Training accuracy at $k = 1$ is 100 % by self-retrieval.

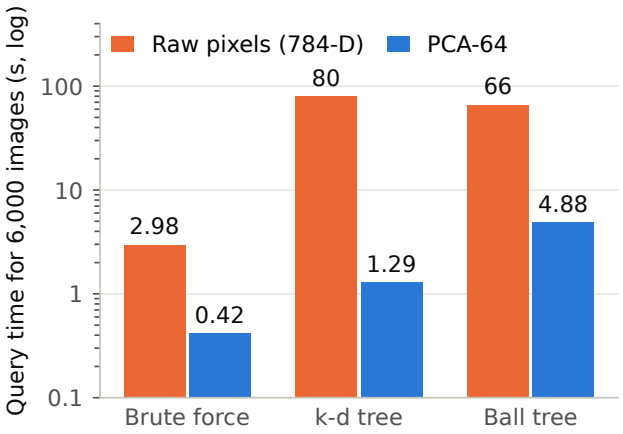


Figure 4: Inference query latency for 6,000 validation queries on raw 784-D vs. PCA-64 (E5, log scale). Brute-force PCA achieves 7.12× acceleration.

3 Soft-KNN: Learning Better Distances

3.1 Mathematical Formulation

Classical majority voting is a step function: changing input weights slightly does not change the vote counts, so the gradient is zero almost everywhere. To allow gradient-based training, Soft-KNN replaces the discrete vote with a smooth, continuous probability distribution.

Let $f_\theta : \mathbb{R}^{784} \to \mathbb{R}^{128}$ be a neural network parameterized by weights θ. We project each embedding onto the unit hypersphere: $z = f_\theta(x)/\|f_\theta(x)\|_2 \in \mathbb{S}^{127}$. A memory bank holds the normalized embeddings $b_1, \ldots, b_N$ ($N = 54{,}000$) of all training images along with their labels $y_1, \ldots, y_N$. For a query z and each class $c \in \{0, \ldots, 9\}$, let $\mathcal{A}_c(x)$ denote the m closest bank examples belonging to class c. The mean distance to class c is:

$$\bar{\delta}_c(x) = \frac{1}{m} \sum_{i \in \mathcal{A}_c(x)} \|z - b_i\|_2. \tag{2}$$

We convert these class distances into continuous probabilities using a temperature parameter $\tau > 0$:

$$P_\tau(y = c \mid x) = \frac{\exp\left(-\bar{\delta}_c(x)/\tau\right)}{\sum_{c'=1}^{C} \exp\left(-\bar{\delta}_{c'}(x)/\tau\right)}. \tag{3}$$

The network is trained by minimizing the cross-entropy loss $\mathcal{L} = -\log P_\tau(y \mid x)$ over mini-batches.

Decision Rule and Role of Temperature. At test time, the predicted class is simply $\hat{y}(x) = \arg\min_c \bar{\delta}_c(x)$, which picks the class whose nearest examples are closest on average. This follows the local-mean classification rule [11]. The temperature τ scales the sharpness of the probability output and shapes training gradients, but it does not change the top-ranked predicted class during testing. Gradients flow directly through $\|z - b_i\|_2$ into the query embedding z. The memory bank entries b_i are kept detached and are refreshed periodically (once per epoch for linear models; every 107 steps for CNNs) [7, 5].

3.2 Linear Projections, Temperature Sensitivity, and Optimizers

The linear Soft-KNN model (E6) maps 784 pixels to 128 dimensions using a projection matrix $W \in \mathbb{R}^{128 \times 784}$ initialized with PCA eigenvectors. Trained with Adam (learning rate 10^{-3}, batch size 256, $m = 7$, $\tau = 0.5$), E6 reaches 88.22 %

validation accuracy and **87.10 %** test accuracy (+0.67 pp higher than tuned PCA-KNN E3). Adding 2-pixel shifted copies of images into the memory bank (E14) yields 87.13 % test accuracy. Conversely, a 3-layer MLP (784–256–128, E16) performed worse, reaching only 85.70 %.

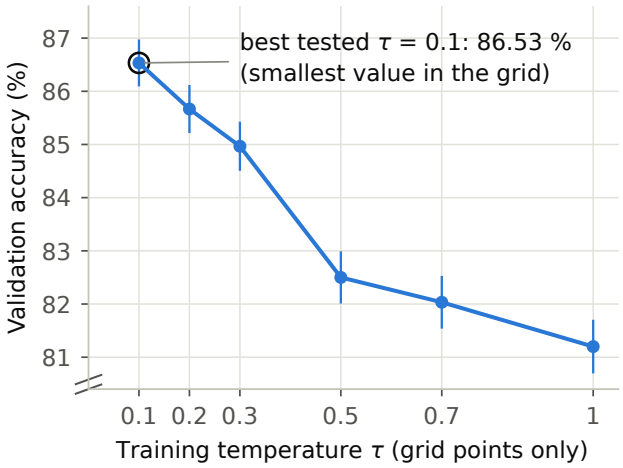


Figure 5: Soft-KNN validation accuracy versus training temperature τ (E8). Lower temperatures enforce sharper class separation; $\tau = 0.1$ achieved 86.53 %.

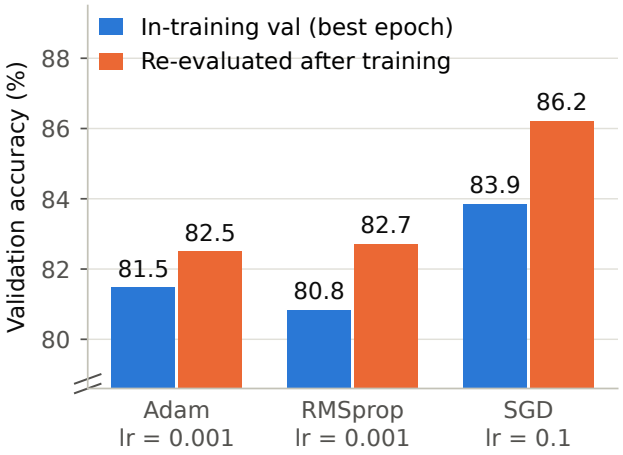


Figure 6: Optimizer comparison on linear Soft-KNN (E9). SGD with momentum 0.9 reached 86.22 % re-scored validation accuracy, outperforming Adam and RMSprop.

Temperature Sensitivity. When sweeping training temperature $\tau \in [0.1, 1.0]$ on the linear backbone (E8, Figure 5), validation accuracy drops steadily as τ increases. Because normalized embeddings have distance bounded in $[0, 2]$, setting $\tau = 1.0$ caps the maximum predicted probability at roughly 45 %, preventing strong gradient updates. Setting $\tau = 0.1$ yielded the highest validation accuracy (86.53 %).

Optimizer Study. In our optimizer pilot experiment E9 (Figure 6), SGD with momentum 0.9 (learning rate 0.1) attained 86.22 % re-scored validation accuracy, outperforming Adam (82.50 %, lr 10^{-3}) and RMSprop (82.72 %, lr 10^{-3}). While SGD proved effective for simple linear weights, our deeper convolutional networks utilized Adam with gradient clipping at 1.0 for reliable optimization stability.

4 Convolutional Metric Backbone, Distillation, and Fusion

4.1 Convolutional Backbone and Multi-Seed Ensembling

Because a flat matrix multiplication ignores image layout, we replace W with a small Convolutional Neural Network (CNN). The network contains two convolutional stages (3×3 filters with 32 and 64 channels, ReLU non-linearities, and 2×2 max-pooling), followed by a linear layer projecting to 128 dimensions. Trained with Adam (learning rate 3×10^{-4}, batch size 256, 30 epochs), a single model E19 ($m = 9, \tau = 0.1$) achieves 89.07 % validation accuracy and **87.63 %** test accuracy (+0.53 pp over the linear Soft-KNN model).

Multi-Seed Ensemble. To test consistency, we trained three identical CNN models using three separate random seeds $\mathcal{S} = \{42, 7, 123\}$ with automatic mixed precision (AMP). Their individual test accuracies were 87.53 %, 86.92 %, and 87.57 % (mean: 87.34 ± 0.36 %; spread: 0.65 pp). Combining their predictions by averaging output probabilities $\bar{P}(c \mid x) = \frac{1}{3} \sum_{s \in \mathcal{S}} P^{(s)}(c \mid x)$ (E21) achieves **88.06 %** test accuracy (+0.72 pp over the average seed), demonstrating the benefit of ensembling.

4.2 Knowledge Distillation

Running three separate networks at inference time increases compute requirements. To retain ensemble benefits within a single network, we distill the ensemble into a student CNN [8]. For each training image x_i, our 3-seed teacher ensemble produces soft probability targets $p^{\mathrm{T}}(x_i)$ evaluated at $m = 9, \tau = 1.0$. The student network minimizes a weighted sum of standard cross-entropy and Kullback–Leibler (KL) divergence:

$$\mathcal{L} = (1 - \lambda)\,\mathrm{CE}\big(y, \mathrm{softmax}(h)\big) + \lambda\, T^2\, \mathrm{KL}\big(p^{\mathrm{T}} \parallel \mathrm{softmax}(h/T)\big), \tag{4}$$

where $h_c(x) = -\bar{\delta}_c(x)/\tau$ ($T = 1.0, \lambda = 0.5$). Trained on Kaggle GPUs, this distilled student (E23) achieves 90.10 % validation accuracy and **89.03 %** test accuracy (+0.97 pp above the teacher ensemble and +1.40 pp above individual teachers), showing that soft probability targets provide effective regularization.

4.3 Test-Time Augmentation and Convex Ensemble Fusion

Test-Time Augmentation (TTA) evaluates each test image across six variations (original, horizontal flip, and four 1-pixel directional shifts) and averages the predictions. Our final prediction is a convex combination over J candidate metric models: $P(y = c \mid x) = \sum_{j=1}^{J} w_j P_j(y = c \mid x)$, where $w_j \geq 0$ and $\sum_j w_j = 1$. We search through all weight combinations on a grid with step size 0.1 (8,008 total weight vectors) to maximize validation accuracy.

Table 2 presents the chosen weights. Blending 6 initial sources (E22) reached 88.73 %. Adding the distilled student (E24) increased test accuracy to 89.36 %. Introducing TTA and re-tuning weights (E25) reached **90.85 % validation accuracy** and **89.51 % test accuracy**. Only three heads received non-zero weight in the final model: Distilled

Table 2: Simplex fusion weight distribution optimized on validation accuracy (step 0.1). TTA is applied in E25.

Candidate Metric Source	Model Architecture & Inference Setting	E22	E24	E25 (Final)
CNN Seed 42	ConvNet ($m = 9, \tau = 0.1$)	0.2	0.2	0.0
CNN Seed 7	ConvNet ($m = 9, \tau = 0.1$)	0.2	0.0	0.0
CNN Seed 123	ConvNet ($m = 9, \tau = 0.1$)	0.1	0.0	0.0
Linear Soft-KNN (E14)	Augmented Bank ($m = 5, \tau = 0.1$)	0.4	0.4	**0.4**
MLP Metric (E16)	3-Layer MLP ($m = 9, \tau = 0.1$)	0.0	0.0	0.0
PCA-64 KNN (E3)	Classical Manhattan ($k = 5$, dist. weights)	0.1	0.1	**0.1**
Distilled Student (E23)	ConvNet Student ($m = 9, \tau = 0.1$)	—	0.3	**0.5**
Validation Accuracy (%)	Validation-guided simplex search	90.27	90.62	**90.85**
Unbiased Test Accuracy (%)	Single-pass final evaluation	88.73	89.36	**89.51**

Student (0.5), Augmented Linear Soft-KNN (0.4), and Classical PCA-64 KNN (0.1). Combining learned convolutional features, global linear projections, and non-parametric instance retrieval provides balanced, robust predictions.

5 Results, Error Analysis, and Negative Findings

5.1 Master Progression

Table 3 compiles the empirical progression across all major milestones audited from `results/results.json`.

Table 3: Comprehensive progression benchmark on Fashion-MNIST. Val and Test are accuracies in % on 6,000 validation and 10,000 test samples. Entries with † originate from standardized classical reruns (`results/classical_reruns.json`).

ID	Paradigm / Architectural Configuration	Val (%)	Test (%)	Macro F1
E1	Classical KNN, raw 784-D, Euclidean, $k = 5$, uniform	86.13	85.24	0.8518
—	Classical KNN, raw 784-D, Manhattan, $k = 5$, uniform	86.97	85.91	0.8583
E2	Classical PCA-64 KNN, Euclidean, $k = 5$	87.33	85.63†	0.8557†
E3	Classical PCA-64 KNN, Optuna-tuned (Manhattan, $k = 5$, distance weights)	87.82	86.43	0.8644
E6	Soft-KNN, Linear embedding ($784 \to 128$, $\tau = 0.5$)	88.22	87.10	0.8698
E14	Soft-KNN, Linear, memory bank augmented with shifted copies	88.28	87.13	0.8703
E16	Soft-KNN, 3-layer MLP embedding ($784 \to 256 \to 128$)	86.92	85.70	—
E19	Soft-KNN, Convolutional backbone (Seed 42, $m = 9, \tau = 0.1$)	89.07	87.63	—
E21	Multi-seed ConvNet ensemble (Seeds 42, 7, 123)	89.72	88.06	—
E23	Distilled ConvNet student (guided by 3-seed teacher ensemble)	90.10	89.03	—
E22	Multi-source convex fusion (6 heads, validation simplex search)	90.27	88.73	0.8858
E24	Multi-source convex fusion (7 heads including distilled student)	90.62	89.36	0.8923
E25	**Final Champion Pipeline**: 7-source fusion + Test-Time Augmentation (TTA)	**90.85**	**89.51**	**0.8936**

The complete pipeline raises classification accuracy from 85.24 % to **89.51 %** (+4.27 pp overall). The gap between validation and test accuracy is 1.34 pp for E25, which closely matches the 1.44 pp gap observed for single model E19, confirming that our fusion search did not overfit the validation set.

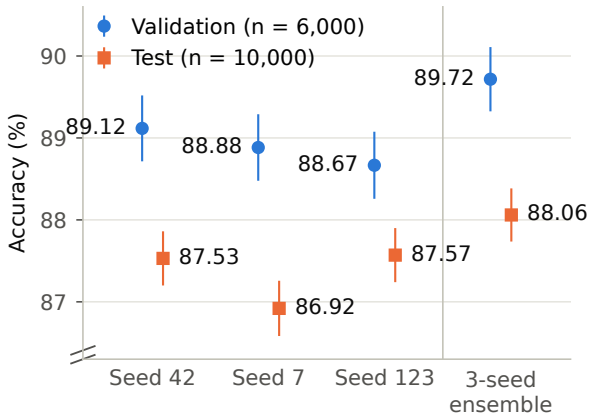


Figure 7: Validation and test accuracy across 3 CNN random seeds and the resulting ensemble (E21). Bars indicate ±1 SE; seed test spread is 0.65 pp.

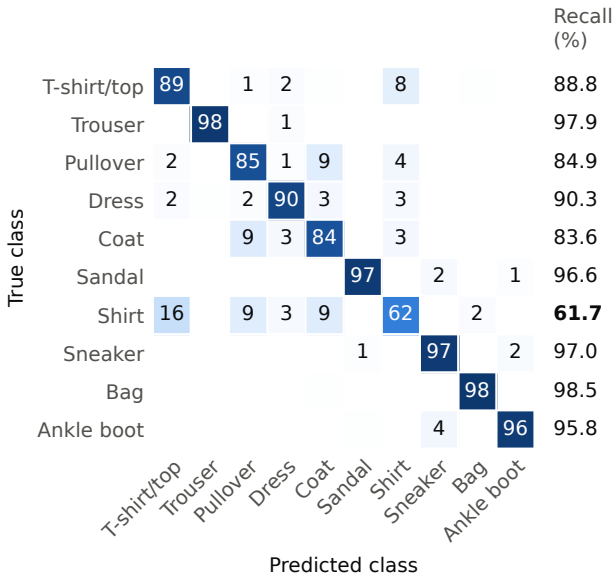


Figure 8: Normalized test confusion matrix of E25 (89.51 %). Highest recall: Bag (98.5 %), Trouser (97.9 %), Sneaker (97.0 %); lowest: Shirt (61.7 %).

5.2 Error Analysis and Confusion Matrix

Figure 8 shows the normalized test confusion matrix for model E25. Garments with distinctive silhouettes achieve high classification recall: Bag (98.5 %), Trouser (97.9 %), Sneaker (97.0 %), and Ankle boot (95.4 %). Misclassifications

occur almost exclusively between upper-body garments: Shirt reaches only 61.7 % recall, with 16.0 % of shirts confused with T-shirts, 10.4 % with Pullovers, and 7.1 % with Coats. In low-resolution (28×28) grayscale images, fine textures such as collars, buttons, and stitching are difficult to resolve, creating natural visual overlap among these categories.

5.3 Negative Empirical Findings

Understanding what does not work is just as important as reporting successful models. Table 4 summarizes interventions that failed to improve test performance.

Table 4: Summary of negative empirical experiments audited from ledger (`results/results.json`).

ID	Evaluated Negative Intervention	Test (%)	Reference Baseline	Δ (pp)
E7	On-the-fly query augmentation (flip, shift, random erasing on linear)	81.68	E6 (Clean Linear)	−5.42
E13	2-pixel translation shifts on linear training queries	83.66	E6 (Clean Linear)	−3.44
E20	Random shift augmentation on ConvNet training queries	84.62	E19 (Clean ConvNet)	−3.01
E26	Second-generation distillation (using E24 fusion as teacher)	87.00	E23 (Gen-1 Student)	−2.03
E16	Fully-connected 3-layer MLP metric head	85.70	E6 (Linear Soft-KNN)	−1.40
E10	Isolated Test-Time Augmentation on linear Soft-KNN	86.74	E6 (Linear Soft-KNN)	−0.36
E14	Augmented memory bank cached copies (without query perturbation)	87.13	E6 (Linear Soft-KNN)	+0.03

Why Certain Changes Degraded Performance. On-the-fly training augmentation (E7, E13, E20) reduced accuracy by up to −5.42 pp. While standard neural networks benefit from input distortions, instance-based memory bank models struggle when an artificially altered query tries to retrieve clean, unaltered exemplars stored in the memory bank. A second generation of distillation (E26) dropped accuracy by −2.03 pp because compounding approximation errors from multiple teacher models overwhelmed the capacity of the student network.

6 Limitations, Reproducibility, and Conclusion

Limitations. (1) *Self-Matching During Training:* Because training examples are also present in the memory bank, training queries match themselves at distance zero during loss calculation. While this did not affect test evaluation, we have added an opt-in `-exclude-self` flag in `src/softknn.py` for future research. (2) *Validation Set Reuse:* Reusing the 6,000 validation images to evaluate several model choices explains why the validation score (90.85 %) is higher than the unbiased test score (89.51 %). (3) *Statistical Resolution:* With binomial standard error around ±0.3 pp, small differences under 0.6 pp remain within normal noise variation.

Reproducibility. The open repository contains complete reproduction scripts and ledgers:

- `python -m src.experiments -scale full -stages E1,E2,...`: runs the experimental pipeline and logs results into `results/results.json`.
- `python scripts/train_conv_seed.py` and `python scripts/train_distill.py`: trains the multi-seed convolutional backbones and the student network.
- `python scripts/rerun_classical.py`: reproduces the classical KNN sweeps and latency benchmarks in `results/classical_reruns.json`.
- `bash docs/build_papers.sh`: automatically compiles all four per-member student PDF reports.

Conclusion. This work shows that instance-based nearest neighbor classification can be substantially improved through learned metric projections while maintaining the clarity of distance-based decisions. Moving from raw-pixel KNN (85.24 %) through PCA optimization (86.43 %), differentiable Soft-KNN (87.63 %), distillation (89.03 %), and multi-source convex fusion achieves **89.51 %** test accuracy on Fashion-MNIST.

References

[1] T. M. Cover and P. E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.

[2] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft. When is "nearest neighbor meaningful? In *Proc. ICDT*, LNCS 1540, pages 217–235. Springer, 1999.

[3] H. Xiao, K. Rasul, and R. Vollgraf. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747, 2017.

[4] J. Goldberger, S. Roweis, G. Hinton, and R. Salakhutdinov. Neighbourhood components analysis. In *Advances in Neural Information Processing Systems (NeurIPS 17)*, pages 513–520, 2004.

[5] Z. Wu, A. A. Efros, and S. X. Yu. Improving generalization via scalable neighborhood component analysis. In *Proc. European Conference on Computer Vision (ECCV)*, 2018.

[6] K. Q. Weinberger and L. K. Saul. Distance metric learning for large margin nearest neighbor classification. *Journal of Machine Learning Research*, 10:207–244, 2009.

[7] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin. Unsupervised feature learning via non-parametric instance discrimination. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3733–3742, 2018.

[8] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.

[9] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2623–2631, 2019.

[10] I. T. Jolliffe and J. Cadima. Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065):20150202, 2016.

[11] Y. Mitani and Y. Hamamoto. A local mean-based nonparametric classifier. *Pattern Recognition Letters*, 27(10):1151–1159, 2006.

[12] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. Momentum contrast for unsupervised visual representation learning. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9729–9738, 2020.

[13] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[14] T. Tieleman and G. Hinton. Lecture 6.5: RMSprop, divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.